3D WEBSOFT INC. FOUNDATION

SCHOOL FUTURE TECH PROGRAM

Faculty Name :

Course Duration : 5 Months (150 Days)

Classes : Offline & Online

Daily Teaching : 3 Hours

Total Hours : 300 Hours

LEVEL 3 –Future Engineers(9th–10th Standard)

MODULE 1 – Python Foundations (Classes 1–10)

Class 9 –OPERATORS IN PYTHON

Complete Theory & Practical Notes

Learning Objectives

By the end of this session, students will be able to:

1. Understand what operators are in Python.
2. Explain the purpose of operators in programming.
3. Identify operands and operators in a Python expression.
4. Understand arithmetic operators.
5. Use +, -, *, /, %, //, and **.
6. Perform basic mathematical calculations using Python.
7. Understand the difference between / and //.
8. Understand the remainder operator %.
9. Use the power operator **.
10. Combine variables and operators to create expressions.
11. Apply operators to real-world problems.
12. Build simple interactive programs using input(), type conversion, variables, and operators.
13. Predict the result of simple Python expressions.
14. Debug common operator-related mistakes.

Session

Topic	*Method*
Warm-up: Mathematics and Programming	Discussion
What Are Operators?	Theory + Demonstration
Arithmetic Operators	Theory + Demonstration
Division `/`, Floor Division `//`, and Modulus `%`	Theory + Practical
Power Operator `**`	Theory + Demonstration
Python Calculator	Hands-on Practical 1:
Break / Operator Challenge	Activity
Expressions, Operands & Operator Precedence	Theory + Demonstration
Shopping Bill Calculator	Hands-on Practical 2
Marks & Average Calculator	Hands-on Practical 3
Quiz + Exit Check	Assessment

1. Warm-up – Mathematics and Programming

Begin by writing these calculations on the board:

```
10 + 5
20 - 8
6 × 4
20 ÷ 5
```

"What are we doing in these examples?"

We are performing calculations.

Explain that computers also need to perform calculations.

For example:

```
Shopping
Banking
Games
School marks
```

```
Engineering
Science
Business
```

all require calculations.

Python uses **operators** to perform these operations.

Real-Life Example

Imagine buying three notebooks.

```
One notebook = ₹40
Quantity = 3
```

The total is:

```
40 × 3 = ₹120
```

In Python:

```
price = 40
quantity = 3

total = price * quantity

print(total)
```

Output:

```
120
```

Introduce the lesson:

Operators are symbols or keywords that tell Python to perform an operation.

2. What Are Operators?

THEORY + DEMONSTRATION

Definition

An **operator** is a symbol or special notation used to perform an operation on values or variables.

Example:

```
10 + 5
```

Here:

```
10 → Operand
+  → Operator
5  → Operand
```

The values being operated on are called **operands**.

Basic Structure

```
Operand    Operator    Operand
   ↓          ↓          ↓
  10          +          5
```

Python evaluates the expression and produces:

```
15
```

Arithmetic Operators

For this beginner class, focus on these arithmetic operators:

Operator	Name	Example	Result
+	Addition	10 + 5	15
-	Subtraction	10 - 5	5
*	Multiplication	10 * 5	50
/	Division	10 / 5	2.0
%	Modulus / Remainder	10 % 3	1
//	Floor Division	10 // 3	3
**	Exponent / Power	2 ** 3	8

These operators allow Python to work like a calculator.

3. Arithmetic Operators

THEORY + DEMONSTRATION

3.1 Addition +

The addition operator adds values.

```
print(10 + 5)
```

Output:

```
15
```

Using Variables

```
a = 20
b = 30

result = a + b

print(result)
```

Output:

```
50
```

Real-Life Example

```
apples = 5
oranges = 3

total_fruits = apples + oranges

print("Total fruits:", total_fruits)
```

Output:

```
Total fruits: 8
```

3.2 Subtraction -

The subtraction operator subtracts one value from another.

```
print(20 - 8)
```

Output:

```
12
```

Example

```
money = 500
spent = 150

remaining = money - spent

print("Remaining:", remaining)
```

Output:

```
Remaining: 350
```

3.3 Multiplication *

Python uses * for multiplication.

```
print(6 * 4)
```

Output:

```
24
```

Example

```
price = 50
quantity = 4

total = price * quantity

print("Total:", total)
```

Output:

```
Total: 200
```

3.4 Division /

Python uses / for division.

```
print(20 / 5)
```

Output:

```
4.0
```

Important observation:

/ produces a division result as a floating-point value.

Example:

```
print(10 / 3)
```

Output:

```
3.3333333333333335
```

Suitable Images – Python Operators

Operator chart:

```
                PYTHON ARITHMETIC OPERATORS

    +         -         *         /         %         //        **
    ↓         ↓         ↓         ↓         ↓         ↓         ↓
  Add     Subtract  Multiply  Divide  Remainder   Floor    Power
```

4. Division /, Floor Division //, and Modulus %

THEORY + PRACTICAL

These three operators are particularly important.

4.1 Division /

```
print(10 / 3)
```

Output:

```
3.3333333333333335
```

Python gives the regular division result.

4.2 Floor Division //

Floor division removes the fractional part by rounding down to the next lower integer.

```
print(10 // 3)
```

Output:

```
3
```

Compare:

```
print(10 / 3)
print(10 // 3)
```

Output:

```
3.3333333333333335
3
```

Simple Explanation

```
/   → Regular division
//  → Floor division
```

4.3 Modulus `%`

The modulus operator gives the **remainder** after division.

```
print(10 % 3)
```

Output:

```
1
```

Why?

```
10 ÷ 3

3 × 3 = 9
10 - 9 = 1
```

Therefore:

```
10 % 3
```

gives:

```
1
```

Even and Odd Numbers

The modulus operator is useful for checking whether a number is even or odd.

Example:

```
number = 10

print(number % 2)
```

Output:

```
0
```

A number divisible by 2 has remainder `0`.

```
number = 7

print(number % 2)
```

Output:

```
1
```

This concept will become especially useful later when students learn **conditional statements**.

Mini Challenge

```
print(15 % 4)
print(20 % 6)
print(25 % 5)
```

Expected:

```
3
2
0
```

*5. Power Operator ***

THEORY + DEMONSTRATION

The `**` operator is used for powers/exponents.

Example:

```
print(2 ** 3)
```

Output:

```
8
```

Because:

```
2 × 2 × 2 = 8
```

More Examples

```
print(5 ** 2)
```

Output:

```
25
```

Because:

```
5 × 5 = 25
```

Another example:

```
print(10 ** 2)
```

Output:

```
100
```

Practical Real-Life Example

Suppose a square has:

```
Side = 5
```

Area:

```
5 × 5 = 25
```

Python:

```
side = 5

area = side ** 2

print("Area:", area)
```

Output:

```
Area: 25
```

6. Python Calculator

HANDS-ON PRACTICAL 1

calculator.py

Starter Code

```
num1 = 20
num2 = 5

print("Addition:", num1 + num2)
print("Subtraction:", num1 - num2)
print("Multiplication:", num1 * num2)
print("Division:", num1 / num2)
print("Floor Division:", num1 // num2)
print("Remainder:", num1 % num2)
print("Power:", num1 ** 2)
```

Expected Output

```
Addition: 25
Subtraction: 15
Multiplication: 100
Division: 4.0
Floor Division: 4
Remainder: 0
Power: 400
```

Student Challenge

Change:

```
num1 = 20
num2 = 5
```

to different values.

Ask students to predict the results before running the program.

7. Break / Operator Challenge

Give students these expressions:

```
8 + 2
10 - 3
4 * 5
20 / 4
17 % 5
17 // 5
```

```
2 ** 4
```

Students calculate the answers **without running Python**.

Then run the code and check.

Expected Answers

Expression	Result
`8 + 2`	`10`
`10 - 3`	`7`
`4 * 5`	`20`
`20 / 4`	`5.0`
`17 % 5`	`2`
`17 // 5`	`3`
`2 ** 4`	`16`

8. Expressions, Operands & Operator Precedence

THEORY + DEMONSTRATION

What is an Expression?

An **expression** is a combination of values, variables, and operators that Python can evaluate.

Example:

```
10 + 5
```

Another example:

```
price * quantity
```

Another:

```
age + 1
```

Operators with Variables

```
price = 100
quantity = 3

total = price * quantity

print(total)
```

Here:

```
price       → variable
quantity    → variable
*           → operator
price * quantity → expression
total       → result variable
```

Operator Precedence

What is the result of:

```
print(10 + 5 * 2)
```

Many beginners may think:

```
(10 + 5) × 2 = 30
```

But Python follows mathematical operator precedence.

Multiplication happens before addition:

```
5 × 2 = 10
10 + 10 = 20
```

Therefore:

```
print(10 + 5 * 2)
```

Output:

```
20
```

Using Parentheses

Parentheses can change the order.

```
print((10 + 5) * 2)
```

Output:

```
30
```

Compare:

```
10 + 5 * 2
```

and:

```
(10 + 5) * 2
```

Results:

```
20
30
```

Beginner Rule

Teach students this simple order:

```
1. Parentheses ()
2. Powers **
3. Multiplication / Division / Floor Division / Modulus
4. Addition / Subtraction
```

For operators at the same level, Python evaluates from left to right.

9. Shopping Bill Calculator

HANDS-ON PRACTICAL 2

Students create:

```
shopping_bill.py
```

Starter Program

```
item_price = 50
quantity = 3

total = item_price * quantity

print("Item Price:", item_price)
print("Quantity:", quantity)
print("Total Bill:", total)
```

Output:

```
Item Price: 50
Quantity: 3
Total Bill: 150
```

Interactive Version

```
item_price = float(input("Enter item price: "))
quantity = int(input("Enter quantity: "))

total = item_price * quantity

print("Total Bill:", total)
```

Example:

```
Enter item price: 75.5
Enter quantity: 4
Total Bill: 302.0
```

Extension – Add a Second Item

```
price1 = float(input("Enter price of item 1: "))
quantity1 = int(input("Enter quantity of item 1: "))

price2 = float(input("Enter price of item 2: "))
quantity2 = int(input("Enter quantity of item 2: "))

total1 = price1 * quantity1
total2 = price2 * quantity2

grand_total = total1 + total2

print("Item 1 Total:", total1)
print("Item 2 Total:", total2)
print("Grand Total:", grand_total)
```

This demonstrates how operators can be combined with variables and user input.

10. Marks & Average Calculator

HANDS-ONPRACTICAL 3

Students create:

```
marks_calculator.py
```

Program

```
maths = float(input("Enter Maths marks: "))
science = float(input("Enter Science marks: "))
english = float(input("Enter English marks: "))

total = maths + science + english
average = total / 3

print()
print("===== MARKS REPORT =====")
print("Total:", total)
print("Average:", average)
```

Example

```
Enter Maths marks: 80
Enter Science marks: 90
Enter English marks: 85

===== MARKS REPORT =====
Total: 255.0
Average: 85.0
```

Student Challenge

Ask students to add two more subjects:

```
computer = float(input("Enter Computer marks: "))
social = float(input("Enter Social Studies marks: "))
```

Then calculate the new total and average.

This reinforces:

```
Input
  ↓
Variables
  ↓
Operators
  ↓
Calculation
  ↓
Output
```

11. Final Operator Challenge

A student has ₹500. They spend ₹175 on books and ₹125 on stationery. Calculate the amount remaining.

Students create:

```
money = 500
books = 175
stationery = 125

remaining = money - books - stationery

print("Money Remaining:", remaining)
```

Expected output:

```
Money Remaining: 200
```

Extension Challenge

A shop sells notebooks for ₹40 each. A student buys 7 notebooks. What is the total cost?

```
price = 40
quantity = 7

total = price * quantity

print("Total Cost:", total)
```

Expected output:

```
Total Cost: 280
```

12. Common Beginner Mistakes

Mistake 1 – Using `x` for multiplication

```
5 x 3
```

Incorrect.

Python uses:

```
5 * 3
```

Mistake 2 – Using / when they need an integer quotient

```
10 / 3
```

produces:

```
3.3333333333333335
```

Whereas:

```
10 // 3
```

produces:

```
3
```

Mistake 3 – Confusing % with percentage

In Python:

```
10 % 3
```

means:

Find the remainder after division.

It does **not** mean “10 percent of 3.”

Mistake 4 – Forgetting parentheses

Incorrect:

```
print 10 + 5
```

Correct:

```
print(10 + 5)
```

Mistake 5 – Forgetting numeric conversion

This can cause problems:

```
num1 = input("Enter number: ")
num2 = input("Enter number: ")

print(num1 + num2)
```

If the user enters:

```
10
20
```

the result is:

```
1020
```

because the values are strings.

For numerical addition:

```
num1 = int(input("Enter number: "))
num2 = int(input("Enter number: "))

print(num1 + num2)
```

Now the result is:

```
30
```

This reinforces the previous **Input & Output** lesson.

13. Classroom Game – "Operator Detective"

Display:

```
20 + 5
20 - 5
20 * 5
20 / 5
20 % 6
20 // 6
2 ** 5
```

Expression	Operator	Meaning
`20 + 5`	`+`	Addition
`20 - 5`	`-`	Subtraction
`20 * 5`	`*`	Multiplication
`20 / 5`	`/`	Division
`20 % 6`	`%`	Remainder
`20 // 6`	`//`	Floor Division

```
2 ** 5                    **                    Power
```

14. Board Summary

At the end of the theory section, write:

```
PYTHON OPERATORS

+    Addition
-    Subtraction
*    Multiplication
/    Division
%    Remainder
//   Floor Division
**   Power
```

Then:

```
Expression:

10 + 5

10 → Operand
+  → Operator
5  → Operand

Result → 15
```

And:

```
INPUT
  ↓
VARIABLES
  ↓
OPERATORS
  ↓
CALCULATION
  ↓
OUTPUT
```

17. Quick Assessment

Question 1

What is an operator?

Answer: An operator is a symbol or notation used to perform an operation on values or variables.

Question 2

Which operator is used for multiplication?

Answer:

```
*
```

Question 3

What is the result of:

```
10 + 20
```

Answer:

```
30
```

Question 4

What is the result of:

```
10 % 3
```

Answer:

```
1
```

Question 5

What is the difference between / and //?

Answer: / performs regular division, while // performs floor division.

Question 6

What does ** do?

Answer: It calculates a power/exponent.

Example:

```
2 ** 3
```

Result:

```
8
```

Question 7

What is the result of:

```
10 + 5 * 2
```

Answer:

```
20
```

because multiplication is performed before addition.

Question 8

What is the result of:

```
(10 + 5) * 2
```

Answer:

```
30
```

because the parentheses are evaluated first.

Learning Outcomes

By the end of this class, students will be able to:

1. **Define** what an operator is in Python.
2. **Identify** operators and operands in simple expressions.

3. **Explain** how operators are used to perform calculations.
4. **Use** the arithmetic operator + for addition.
5. **Use** the arithmetic operator - for subtraction.
6. **Use** the arithmetic operator * for multiplication.
7. **Use** the arithmetic operator / for division.
8. **Use** the modulus operator % to find remainders.
9. **Use** floor division // for floor-division results.
10. **Use** the power operator ** for exponent calculations.
11. **Create** expressions using variables and arithmetic operators.
12. **Differentiate** between regular division / and floor division //.
13. **Explain** how % can be used to find remainders.
14. **Apply** operator precedence and parentheses in expressions.
15. **Combine** input(), variables, type conversion, operators, and print() in one program.
16. **Create** a basic calculator program.
17. **Create** a shopping bill calculator.
18. **Create** a marks and average calculator.
19. **Predict** the results of simple Python expressions before running them.
20. **Identify and correct** common beginner mistakes involving arithmetic operators.